

Table des matières

1. Pour commencer
2. Retour sur les expressions
3. Les expressions booléennes

1. Pour commencer

Pour [le projet](#), nous avons besoin de mettre en mémoire les instructions analysées pour les exécuter ensuite (pour pouvoir exécuter des fonctions, par exemple). Dans ce TP, nous allons mettre en place la création d'arbres d'évaluation pour stocker en mémoire les expressions.

2. Retour sur les expressions

Pour cet exercice, nous nous basons sur la calculatrice *Lex&Yacc* basique : elle doit reconnaître les opérations de base ('-', '+', '*', '/'), le moins unaire, ainsi que les parenthèses. Il faut gérer les problèmes de priorité. Plusieurs opérations de suite doivent être reconnues.

1. Écrivez ce programme en vous basant sur le code du TP précédent.
2. Vérifiez que toutes les opérations sont bien reconnues.

L'objectif est de partir de la grammaire et de modifier sa décoration pour que le calcul ne soit plus effectué au fur-et-à-mesure, mais qu'il soit mis en mémoire sous la forme d'un arbre. Une fois construit, il peut être évalué (au niveau de l'axiome).

Le plus simple est de créer une structure `tree_t` caractérisée par un type, une valeur et des fils. Sachant qu'une expression peut être constituée d'un appel de fonction, le nombre de fils peut être variable. Il est donc conseillé de créer une liste d'arbres (`tree_list_t`) pour représenter les fils. Pour la valeur, il est plus simple d'utiliser une union plutôt qu'un pointeur générique.

3. Quels sont les types possibles pour une expression ?

Spoil alert

4. Proposez une structure pour représenter un arbre.

Saviez-vous qu'il est possible d'utiliser une union dans une structure sans lui donner de nom (on parle d'union anonyme) ? Dans ce cas, il est possible d'accéder directement aux champs de l'union.

Par exemple :

```
typedef struct {  
    int i;  
    union {  
        int j;  
        char *tmp;  
    }  
} exemple_t;  
...  
exemple_t tmp;  
tmp.j = 2;  
...
```

5. Créez un programme de test pour vérifier que toutes vos fonctions sont fonctionnelles.
6. Modifiez la grammaire pour créer l'arbre au fur-et-à-mesure. Affichez-le puis détruisez-le au niveau de l'axiome.
7. Ajoutez, si ce n'est pas déjà fait, la possibilité d'évaluer l'arbre.

3. Les expressions booléennes

Nous allons maintenant gérer les expressions booléennes en plus des expressions entières.

1. Est-ce que des modifications sont à prévoir pour votre structure d'arbre ?
2. Quels sont les opérateurs nécessaires spécifiques aux booléens ? Modifiez votre fichier *Lex* (et n'oubliez pas d'ajouter les constantes pour *vrai* et *faux*).
3. Proposez une grammaire pour construire des arbres d'expressions booléennes en plus des expressions entières.
4. Vérifiez que vous pouvez analyser et évaluer $12+8==2*10$.